Formulario de registro

Esta guía arma el formulario dentro de `RegisterScreen`. Hoy esa pantalla solo muestra el texto "Pantalla de registro". La ruta `register_page` y el `NavHost` no se tocan.

Un formulario es un grupo de campos donde la persona escribe datos. Aquí los datos son el nombre, el correo y una casilla de términos. La pantalla no guarda esos datos por su cuenta: se los pide a un `ViewModel`, que es la clase encargada de recordarlos y de revisar si están bien.

La materia separa el trabajo en tres partes:

- El **modelo** es la ficha con los datos y los mensajes de error.
- El **ViewModel** actualiza esa ficha cada vez que alguien escribe o pulsa Enviar.
- La **pantalla** solo dibuja lo que dice la ficha. Si la ficha cambia, la pantalla se vuelve a dibujar sola.

1. El modelo

Crea el paquete `cl.duoc.myapplication.model` y el archivo `RegistroUiState.kt`:

```
package cl.duoc.myapplication.model

data class RegistroUiState(
    val nombre: String = "",
    val correo: String = "",
    val aceptaTerminos: Boolean = false,
    val errores: RegistroErrores = RegistroErrores()
)

data class RegistroErrores(
    val nombre: String? = null,
    val correo: String? = null,
    val aceptaTerminos: String? = null
)
```

Una `data class` es una clase hecha para guardar datos. En vez de crear variables sueltas (`nombre` por un lado, `correo` por otro), las dejamos juntas en un solo objeto. Así, cuando algo cambia, viaja la ficha completa y no una variable perdida.

`RegistroUiState` es esa ficha. Cada propiedad es un campo del formulario:

- `nombre` y `correo` son `String`, o sea texto. Empiezan en `""`, un texto vacío, porque al abrir la pantalla la persona todavía no escribió nada.
- `aceptaTerminos` es `Boolean`: solo puede ser `true` (marcado) o `false` (sin marcar). Empieza en `false`.
- `errores` es otra ficha, `RegistroErrores`, metida adentro. Ahí van los mensajes de "este campo está mal".

El signo `?` en `String?` significa que el valor puede ser un texto o puede no existir. `null` quiere decir "no hay mensaje". Si `errores.nombre` es `null`, el nombre está bien y no hay que mostrar nada debajo del campo. Si es `"El nombre es obligatorio"`, ese texto es el aviso.

Agrupar datos y errores en la misma ficha permite que el `ViewModel` publique un único estado. La pantalla no tiene que preguntar campo por campo: recibe la ficha nueva y se redibuja.

2. El ViewModel

Crea `viewmodels/RegistroViewModel.kt`.

Ya existe un `MainViewModel`, pero ese solo sirve para cambiar de pantalla. Este es otro: se llama `RegistroViewModel` y su trabajo es recordar lo que se escribe en el formulario y decir si es válido.

```kotlin
package cl.duoc.myapplication.viewmodels

import androidx.lifecycle.ViewModel
import cl.duoc.myapplication.model.RegistroErrores
import cl.duoc.myapplication.model.RegistroUiState
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update

class RegistroViewModel : ViewModel() {
    private val _estado = MutableStateFlow(RegistroUiState())
    val estado: StateFlow<RegistroUiState> = _estado.asStateFlow()

    fun onNombreChange(valor: String) {
        _estado.update {
            it.copy(
                nombre = valor,
                errores = it.errores.copy(nombre = null)
            )
        }
    }

    fun onCorreoChange(valor: String) {
        _estado.update {
            it.copy(
                correo = valor,
                errores = it.errores.copy(correo = null)
            )
        }
    }

    fun onAceptarTerminosChange(valor: Boolean) {
        _estado.update {
            it.copy(
                aceptaTerminos = valor,
                errores = it.errores.copy(aceptaTerminos = null)
            )
        }
    }

    fun validar(): Boolean {
        val actual = _estado.value
        val errores = RegistroErrores(
            nombre = if (actual.nombre.isBlank()) "El nombre es obligatorio" else null,
            correo = when {
                actual.correo.isBlank() -> "El correo es obligatorio"
                !actual.correo.contains("@") -> "El correo debe incluir @"
                else -> null
            },
            aceptaTerminos = if (!actual.aceptaTerminos) "Debes aceptar los términos" else
        null
        )
        _estado.update { it.copy(errores = errores) }
        return errores.nombre == null && errores.correo == null && errores.aceptaTerminos ==
        null
    }
}
```

`ViewModel()` al final de la primera línea dice que esta clase hereda de `ViewModel`. Eso la mantiene viva aunque la pantalla se redibuje, por ejemplo al girar el teléfono. Los textos que la persona escribió no se pierden.

`_estado` es la ficha privada. El guion bajo avisa que nadie de afuera debe escribir ahí. `MutableStateFlow` es una caja observable: guarda un valor y avisa a quien esté mirando cada vez que ese valor cambia. Al crearla con `RegistroUiState()`, la ficha parte vacía.

`estado` es la misma caja, pero de solo lectura (`StateFlow`). La pantalla puede mirarla. No puede reemplazarla por su cuenta. `asStateFlow()` hace esa conversión.

`update` cambia la ficha. Dentro de las llaves, `it` es la ficha actual. `copy` crea una ficha nueva igual a la anterior, cambiando solo lo que se indica. No se modifica la ficha vieja: se publica una nueva, y por eso la pantalla se entera del cambio.

`onNombreChange` recibe el texto nuevo del campo. Copia la ficha con ese nombre y, al mismo tiempo, pone `errores.nombre` en `null`. Si había un mensaje de error, desaparece en cuanto la persona empieza a corregir. `onCorreoChange` y `onAceptarTerminosChange` hacen lo mismo con su campo.

`validar` se llama al pulsar Enviar. Lee la ficha actual y arma una ficha de errores nueva:

- Si el nombre está en blanco (`isBlank()` también cuenta los espacios), el mensaje es “El nombre es obligatorio”. Si tiene texto, el error queda en `null`.
- El `when` revisa el correo en orden. Primero si está vacío. Si no, si le falta el `@`. Si pasa las dos pruebas, no hay error.
- Si la casilla no está marcada, avisa que hay que aceptar los términos.

Después publica esa ficha de errores dentro del estado, para que los campos puedan mostrarla. Al final devuelve `true` solo cuando los tres errores son `null`. `true` significa “se puede enviar”. `false` significa “todavía hay algo mal”.

3. La pantalla, de a poco

`RegisterScreen` se va armando en el mismo archivo. Después de cada bloque, abre la pantalla de registro en la app. Así se ve qué agregó ese pedazo.

3.1 Observar el estado

Reemplaza `ui/RegisterScreen.kt` por esto:

```
package cl.duoc.myapplication.ui

import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewmodel.compose.viewModel
import cl.duoc.myapplication.viewmodels.RegistroViewModel

@Composable
fun RegisterScreen(viewModel: RegistroViewModel = viewModel()) {
    val estado by viewModel.estado.collectAsState()

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
    ) {
        Text("Nombre: ${estado.nombre}")
    }
}
```

`@Composable` marca la función como una pieza de interfaz. Compose la dibuja cuando el `NavHost` entra a `register_page`.

`viewModel: RegistroViewModel = viewModel()` pide el `ViewModel` del formulario. `viewModel()` lo crea la primera vez y, si la pantalla se redibuja, devuelve el mismo. El `= viewModel()` es un valor por defecto: quien llama a `RegisterScreen()` no tiene que pasar nada. Por eso la línea del `NavHost` sigue igual.

`collectAsState()` se suscribe a la caja `estado`. Cada vez que el `ViewModel` publica una ficha nueva, Compose vuelve a ejecutar esta función. `by` permite escribir `estado.nombre` en vez de `estado.value.nombre`.

La `Column` apila lo que pongas dentro, de arriba hacia abajo. `fillMaxSize` la hace ocupar toda la pantalla y `padding(16.dp)` deja 16 dp de margen. `dp` es la unidad de espacio en Android: se ve parecido en pantallas distintas.

El `Text` provisional lee el nombre de la ficha. Como todavía está vacío, en pantalla se lee “Nombre:”. Sirve para comprobar que la pantalla ya está mirando el `ViewModel` antes de poner los campos.

3.2 El campo nombre

Agrega estos imports:

```
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.material3.OutlinedTextField
```

Un `import` le dice a Kotlin en qué librería está una clase. Sin él, el archivo no reconoce `OutlinedTextField`.

Dentro de la `Column`, reemplaza el `Text` por el campo:

```
OutlinedTextField(
    value = estado.nombre,
    onValueChange = viewModel::onNombreChange,
    label = { Text("Nombre completo") },
    modifier = Modifier.fillMaxWidth()
)
```

`OutlinedTextField` es el cuadro de texto con borde de Material. La persona escribe ahí.

- `value` es el texto que se muestra. Sale de la ficha, no de una variable local de la pantalla.
- `onValueChange` se ejecuta en cada tecla. `viewModel::onNombreChange` es un atajo para decir "llama a esa función y pásale el texto nuevo". El `ViewModel` publica otra ficha y el campo se vuelve a dibujar con la letra recién escrita.
- `label` es el nombre del campo, el texto gris que se ve dentro o arriba del borde.
- `fillMaxWidth` estira el campo al ancho de la columna.

Eso es la vista en tiempo real: no hay que pulsar un botón para que el texto quede guardado. Cada letra ya está en la ficha.

3.3 El correo y el error del campo

Debajo del nombre, agrega el correo. Este campo ya sabe mostrar el error que guarda la ficha:

```
OutlinedTextField(
    value = estado.correo,
    onValueChange = viewModel::onCorreoChange,
    label = { Text("Correo electrónico") },
    isError = estado.errores.correo != null,
    supportingText = {
        estado.errores.correo?.let { mensaje ->
            Text(mensaje)
        }
    },
    modifier = Modifier.fillMaxWidth()
)
```

Funciona igual que el nombre, con dos datos extra:

- `isError` es un sí o un no. `!= null` pregunta "¿hay un mensaje de error?". Si lo hay, Material pinta el borde de rojo.
- `supportingText` es el texto de ayuda que va debajo del campo. `?.let` significa: si `errores.correo` no es `null`, entra al bloque y usa ese texto como `mensaje`. Si es `null`, no dibuja nada.

Agrega el mismo par en el campo nombre, para que también pueda avisar:

```
isError = estado.errores.nombre != null,
supportingText = {
    estado.errores.nombre?.let { mensaje ->
        Text(mensaje)
    }
},
```

Esas dos líneas van dentro del `OutlinedTextField` del nombre, junto a `label`. El mensaje no aparece al abrir la pantalla, porque los errores parten en `null`. Aparece cuando alguien pulsa Enviar y `validar()` encuentra el campo vacío. Si después la persona escribe, `onNombreChange` borra ese error y el borde rojo se va.

3.4 Los términos

Agrega estos imports:

```
import androidx.compose.foundation.layout.Row
import androidx.compose.material3.Checkbox
import androidx.compose.ui.Alignment
```

Debajo de los campos:

```
Row(verticalAlignment = Alignment.CenterVertically) {
    Checkbox(
        checked = estado.aceptaTerminos,
        onCheckedChange = viewModel::onAceptarTerminosChange
    )
    Text("Acepto los términos")
}

estado.errores.aceptaTerminos?.let { mensaje ->
    Text(mensaje)
}
```

Una `Row` pone los hijos uno al lado del otro, en horizontal. `CenterVertically` alinea la casilla y el texto a la misma altura, para que el texto no quede más arriba que el cuadradito.

`Checkbox` es una casilla de dos estados: marcada o sin marcar. `checked` lee `aceptaTerminos` de la ficha. `onCheckedChange` avisa al `ViewModel` cuando la persona la toca, y la ficha pasa a `true` o a `false`.

El `Text` de abajo usa el mismo `?.let` del correo. Solo se dibuja si la validación dejó un mensaje en `aceptaTerminos`.

3.5 La vista previa en tiempo real

Agrega este import:

```
import androidx.compose.material3.HorizontalDivider
```

Debajo del checkbox:

```
HorizontalDivider(modifier = Modifier.padding(vertical = 16.dp))

Text("Vista previa en tiempo real")
Text("Nombre: ${estado.nombre}")
Text("Correo: ${estado.correo}")
Text(
    "Términos aceptados: ${if (estado.aceptaTerminos) "Sí" else "No"}"
)
```

`HorizontalDivider` es una línea horizontal. Separa el formulario de la vista previa para que se lean como dos bloques. `padding(vertical = 16.dp)` deja espacio arriba y abajo de la línea.

Los tres `Text` leen la misma ficha que los campos. `${estado.nombre}` inserta el valor dentro de la frase. El `if` elige “Sí” o “No” según la casilla, porque un `Boolean` en pantalla se vería como `true` o `false`, y “Sí” o “No” se entiende mejor.

Al escribir o marcar la casilla, estos textos cambian al instante. No hace falta pulsar Enviar: la pantalla ya está suscrita a la ficha desde el paso 3.1.

3.6 Enviar y avisar el resultado

Agrega estos imports:

```
import androidx.compose.material3.Button
import androidx.compose.material3.SnackbarHost
import androidx.compose.material3.SnackbarHostState
import androidx.compose.runtime.remember
import androidx.compose.runtime.rememberCoroutineScope
import kotlinx.coroutines.launch
```

Arriba de la `Column`, junto al `estado`, crea el aviso y el alcance para mostrarlo:

```
val snackbarHostState = remember { SnackbarHostState() }
val scope = rememberCoroutineScope()
```

Un `Snackbar` es la barrita de aviso que aparece un momento, en este caso debajo del botón. `SnackbarHostState` es el objeto que guarda el mensaje que se va a mostrar.

`remember` guarda ese objeto entre redibujados. Sin `remember`, cada tecla crearía un aviso nuevo y el mensaje se perdería.

`showSnackbar` es una función que se suspende: espera a que el aviso termine de mostrarse. Por eso no se puede llamar directo dentro de `onClick`. `rememberCoroutineScope()` entrega un lugar, `scope`, desde donde sí se puede lanzar con `launch`.

Al final de la `Column`, el botón y el lugar donde aparece el aviso:

```
Button(
    onClick = {
        if (viewModel.validar()) {
            scope.launch {
                snackbarHostState.showSnackbar("Formulario enviado correctamente")
            }
        } else {
            scope.launch {
                snackbarHostState.showSnackbar("Errores detectados. Revisa los campos.")
            }
        }
    }
) {
    Text("Enviar")
}

SnackbarHost(hostState = snackbarHostState)
```

`onClick` corre al pulsar el botón. Lo primero que hace es `validar()`. Esa función revisa los tres campos y deja los mensajes en la ficha. Como la pantalla observa la ficha, los bordes rojos y los textos de error aparecen en ese momento.

Si `validar()` devuelve `true`, el aviso dice que el formulario se envió. Si devuelve `false`, el aviso pide revisar los campos. Los dos caminos usan `scope.launch` porque mostrar el aviso es una espera.

`SnackbarHost` es el hueco donde ese aviso se dibuja. Sin él, el mensaje queda guardado y no se ve.

Cómo queda el archivo

Este es `ui/RegisterScreen.kt` completo, por si al armarlo de a poco se perdió alguna línea. Va en dos partes para que cada bloque quepa en una hoja: primero los imports y el campo nombre, después el resto de la columna.

Parte 1:

```kotlin
package cl.duoc.myapplication.ui

import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.Checkbox
import androidx.compose.material3.HorizontalDivider
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.SnackbarHost
import androidx.compose.material3.SnackbarHostState
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.remember
import androidx.compose.runtime.rememberCoroutineScope
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewmodel.compose.viewModel
import cl.duoc.myapplication.viewmodels.RegistroViewModel
import kotlinx.coroutines.launch

@Composable
fun RegisterScreen(viewModel: RegistroViewModel = viewModel()) {
    val estado by viewModel.estado.collectAsState()
    val snackbarHostState = remember { SnackbarHostState() }
    val scope = rememberCoroutineScope()

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
    ) {
        OutlinedTextField(
            value = estado.nombre,
            onValueChange = viewModel::onNombreChange,
            label = { Text("Nombre completo") },
            isError = estado.errores.nombre != null,
            supportingText = {
                estado.errores.nombre?.let { mensaje ->
                    Text(mensaje)
                }
            },
            modifier = Modifier.fillMaxWidth()
        )
```

Parte 2, a continuación dentro de la misma `Column`:

```
        OutlinedTextField(
            value = estado.correo,
            onValueChange = viewModel::onCorreoChange,
            label = { Text("Correo electrónico") },
            isError = estado.errores.correo != null,
            supportingText = {
                estado.errores.correo?.let { mensaje ->
                    Text(mensaje)
                }
            },
            modifier = Modifier.fillMaxWidth()
        )

        Row(verticalAlignment = Alignment.CenterVertically) {
            Checkbox(
                checked = estado.aceptaTerminos,
                onCheckedChange = viewModel::onAceptarTerminosChange
            )
            Text("Acepto los términos")
        }

        estado.errores.aceptaTerminos?.let { mensaje ->
            Text(mensaje)
        }

        HorizontalDivider(modifier = Modifier.padding(vertical = 16.dp))

        Text("Vista previa en tiempo real")
        Text("Nombre: ${estado.nombre}")
        Text("Correo: ${estado.correo}")
        Text("Términos aceptados: ${if (estado.aceptaTerminos) "Sí" else "No"}")

        Button(
            onClick = {
                if (viewModel.validar()) {
                    scope.launch {
                        snackbarHostState.showSnackbar("Formulario enviado correctamente")
                    }
                } else {
                    scope.launch {
                        snackbarHostState.showSnackbar("Errores detectados. Revisa los
    campos.")
                    }
                }
            }
        ) {
            Text("Enviar")
        }

        SnackbarHost(hostState = snackbarHostState)
    }
}
```

Qué probar

1. En el inicio, pulsa **Registrarse**. Deben verse los dos campos, la casilla y la vista previa vacía.
2. Pulsa **Enviar** sin escribir nada. El nombre, el correo y los términos deben mostrar su mensaje, y la barrita de aviso debe pedir revisarlos.

3. Escribe un nombre. El error del nombre desaparece al tipear, y la vista previa muestra ese nombre al mismo tiempo.
4. Escribe un correo sin `@` y pulsa **Enviar**. El correo queda marcado como error.
5. Escribe un correo con `@` y marca los términos. Al pulsar **Enviar**, el aviso dice que el formulario se envió.